Trucos de manipulación de bits

Pregunta	Expresión	Ejemplo
1- ¿ Comprueba si un entero es **par o impar** ?	**& (AND)** **N & 1** devuelve 1 o 0 dependiendo de si n es par o impar.	00010100 & (n = 20) 00000001 (1) ~~~~~~~~ 00000000 00010101 & (n = 21) 00000001 (1) ~~~~~~~~ 00000001

```
#include <iostream>
using namespace std;
int main()
{
   int n = 5;
   if (n & 1) {
      cout << n << " is odd";
   }
   else {
      cout << n << " is even";
   }
   return 0;
}
```

Nota:

- Tener en cuenta que el último bit(**bit más significativo** - **MSB**) de un binario dice 0=par, 1=impar
- Recuerda que c++, te brinda la posibilidad trabajar con números de dos formas:

1. **Como números normales**: los ve enteros, como 5, 12, 100, y hace operaciones normales (+, -, *, /).
2. **Como números binarios**: los ve como una fila de 0 y 1. Por tanto te permite hacer operaciones **bit a bit** (bitwise), que actúan sobre cada 0 o 1 individualmente, en lugar de usar el número completo. Entonces tu en este caso estas trabajando los números como binario, al usar operadores binario.

Pregunta	Expresión	Ejemplo
¿Detectar si dos números enteros tienen **signos opuestos** o no?	**^ (XOR)** **((x ^ y) < 0)** es negativa si x y y tienen signos opuestos **positivo**:bit más a la izquierda **0** **negativo:**bit más a la izquierda **1** **Entero de signos iguales:** XOR establece bit más a la izquierda **0** **Entero de signos opuestos:** XOR establece bit más a la izquierda **1**	**0**0…000100 ^ (x = 4) **0**0…001000 (y = 8) ~~~~~~~~~~ **0**0…001100 igual signo 00…000100 ^ (x = 4) 11…111000 (y = -8) ~ ~~~~~~~~~ 11…111100 diferente signo 10…000100 ^ (x = 4)

		11…111000 (y = -8) ~ ~~~~~~~~ 01…111100 igual signo

```
#include <iostream>
#include <bitset>
using namespace std;

int main()
{
   int x = 4;
   int y = -8;

   //bitset<32>(x) convierte el entero x en una secuencia de 32 bits (0s y 1s).
   //cout << ... imprime en la consola ejemplo: 5 en binario es 00000000000000000000000000000101
   cout << x << " en binario es " << bitset<32>(x) << endl;
   cout << y << " en binario es " << bitset<32>(y) << endl;


   bool isOpposite = ((x ^ y) < 0);  // true si “x” e “y” tienen signo opuesto

   if (isOpposite) {
      cout << x << " and " << y << " tienen signo opuesto";
   }
   else {
      cout << x << " and " << y << " tienen signos iguales";
   }

   return 0;
}
```

La salida de la expresión `x ^ y` es **negativa** si `x` e `y` tienen **signos opuestos**. Sabemos que, para un número **positivo**, el **bit más a la izquierda(bit menos significativo – LSB) es** `0` y, para un número **negativo**, es `1`. Ahora bien, para enteros de **signos iguales**, el operador XOR establecerá el bit más a la izquierda de la salida como **0**, y para enteros de **signo opuesto**, lo establecerá como **1**.

En C++, los números enteros negativos se representan usando **complemento a 2**. Esto significa que el **bit más a la izquierda** (el bit de mayor peso, también llamado MSB) se usa para indicar el signo:

- `0` → positivo
- `1` → negativo

Cuando haces un **XOR** (^) entre dos números:

1. Si un número es negativo y el otro positivo, el XOR dejará un `1` en el bit más significativo, porque los bits de signo son diferentes.
2. Si ambos son negativos o ambos positivos, el XOR no tendrá `1` en el bit de signo, porque son iguales y e

XOR cancela los bits iguales.

Por eso, después de hacer `x ^ y`:

- Si el resultado tiene un `1` en el bit más a la izquierda (`(x ^ y) < 0`), eso significa que **los números tienen signos diferentes**, y si estabas comparando, el resultado será **menor que cero**.
- Si el resultado no tiene ese `1`, los números tienen el mismo signo y el XOR no indica diferencia de signo

Pregunta	**Expresión**	**Ejemplo**
Sumar 1 a un número entero	~ (NOT): Invierte el valor de los bit y suma 1 Los números negativos se almacenan en forma de complemento a 2 ~x= -(x + 1) ejemplo: ~2 = -3 -~x= (x + 1) ejemplo: -~2 = 3	Ejemplo: ~2 = -3 Ejemplo: -~2 = 3 00000010 ~ (x = 2) **1**1111101(C_2) (x = -3) **1**1111101(C_2) = 00000011($_{10}$) (x = -3)

Pregunta	**Expresión**	**Ejemplo**
Intercambiar dos números sin usar ninguna tercera variable	x ^ x = 0 . **x = x ^ y;** **y = x ^ y;** **x = x ^ y;**	00010100 ^ (x = 20) 00010101 (y = 21) ~~~~~~~~ 00000001 **x = x ^ y;** **(diferencia)** 00000001 ^ (x = 1) 00010101 (y = 21) ~~~~~~~~ 00010100 **y = x ^ y;** 00000001 ^ (x = 1) **00010100** (y = 21) ~~~~~~~~ **00010101** **x = x ^ y;**